

# 图书借阅管理系统

## C++ 面向对象课程设计项目报告

项目名称 图书借阅管理系统  
学生姓名 于杭  
学 号 1030425211  
班 级 计科 2502  
指导教师 无  
提交日期 2026 年 7 月

讲解视频 [https://youtu.be/o\\_A9u5vabC4](https://youtu.be/o_A9u5vabC4)

GitHub 仓库 <https://github.com/hhh6996/cpp-library-management-system>

# 目录

|                      |          |
|----------------------|----------|
| <b>1 项目概述</b>        | <b>2</b> |
| 1.1 项目背景与目标          | 2        |
| 1.2 主要功能             | 2        |
| 1.3 运行环境与启动方式        | 2        |
| 1.4 运行截图             | 3        |
| <b>2 面向对象设计详解</b>    | <b>4</b> |
| 2.1 需求分析             | 4        |
| 2.1.1 项目描述           | 4        |
| 2.1.2 功能需求           | 4        |
| 2.1.3 用例分析           | 5        |
| 2.2 总体设计             | 5        |
| 2.2.1 类继承结构图         | 5        |
| 2.2.2 类职责说明          | 6        |
| 2.2.3 继承策略分析         | 6        |
| 2.3 详细设计             | 6        |
| 2.3.1 关键类的接口设计       | 6        |
| 2.3.2 虚函数与多态的体现      | 7        |
| 2.3.3 核心业务逻辑：借阅-归还流程 | 7        |
| <b>3 核心难点与解决方案</b>   | <b>8</b> |
| 3.1 问题描述             | 8        |
| 3.2 定位过程             | 8        |
| 3.3 解决方案             | 8        |
| <b>4 编译与运行说明</b>     | <b>8</b> |
| 4.1 开发环境与编译          | 8        |
| 4.2 运行示例             | 9        |
| <b>5 总结与心得</b>       | <b>9</b> |
| 5.1 项目完成情况           | 9        |
| 5.2 对面向对象设计的理解深化     | 9        |
| 5.3 收获与改进方向          | 9        |
| <b>A 附录：完整源代码</b>    | <b>9</b> |
| A.1 文件清单             | 9        |
| A.2 头文件              | 9        |
| A.3 实现文件             | 13       |

## 1 项目概述

### 1.1 项目背景与目标

图书馆是校园生活中最常见的场景之一。不同类型的读者（学生、教师）享有不同的借阅权限，不同类型的馆藏（图书、杂志）也有不同的借阅规则和逾期罚款标准。传统的纸质管理方式无法高效处理这些差异化的规则。

本项目的目标是开发一个基于控制台的图书借阅管理系统，通过 C++ 面向对象编程技术，利用继承和多态机制，实现对不同读者类型和不同馆藏类型的统一管理。系统支持图书与杂志的增删查改、学生与教师的注册管理、借书还书操作以及逾期罚款的自动计算。

### 1.2 主要功能

**图书管理：**添加图书（书名/ISBN/作者/出版社/年份/页数）、添加杂志（刊名/刊号/主编/年份/期号/月份）、按 ISBN 删除（自动检查借阅记录）、按关键词模糊搜索、显示全部馆藏。

**读者管理：**添加学生（自动设定借阅上限 5 本、借期 30 天）、添加教师（自动设定借阅上限 10 本、借期 60 天）、按编号删除（自动检查借阅记录）、显示全部读者。

**借阅管理：**借书（输入读者编号和物品 ISBN，自动检查借阅数量上限和库存状态）、还书（输入 ISBN，自动计算逾期天数和罚款金额）、查看全部借阅记录、逾期记录查询。

### 1.3 运行环境与启动方式

**操作系统：**Windows 10/11。**编译器：**g++ (MinGW-w64) 或 Visual Studio 2022 (MSVC)。C++ 标准：C++11。无第三方库依赖。

**启动方式：**(1) VS2022 打开 CMakeLists.txt，Ctrl+F5 运行；(2) 命令行执行 `g++ -std=c++11 -o library.exe *.cpp && ./library.exe`。



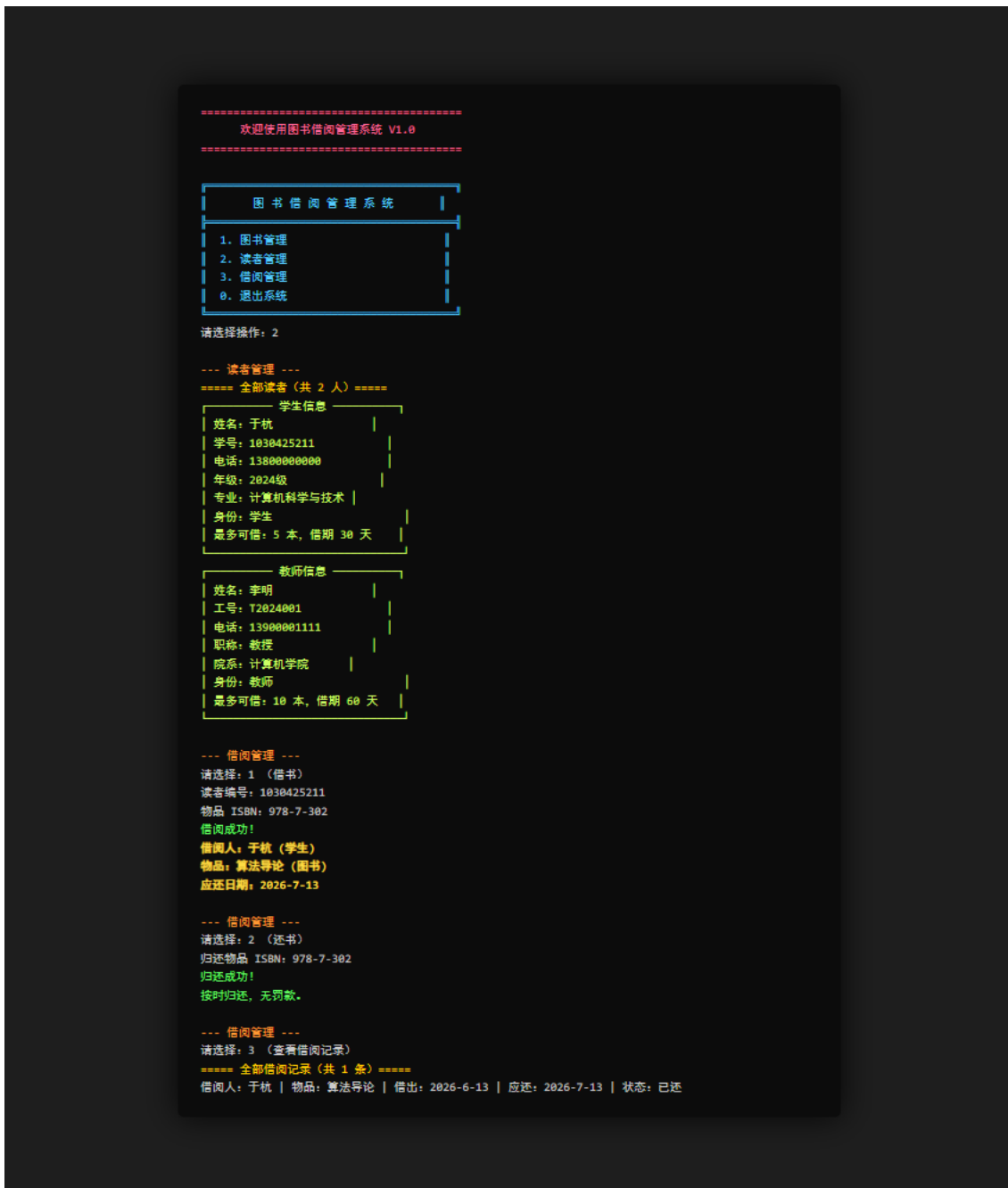


图 2: 学生于杭借阅《算法导论》: 系统自动识别学生身份设定借期 30 天; 还书自动计算 (无逾期); 查看借阅记录确认状态。

## 2 面向对象设计详解

### 2.1 需求分析

#### 2.1.1 项目描述

本系统模拟高校图书馆的日常借阅业务。核心场景: 图书馆拥有多种类型馆藏 (图书和杂志), 面向不同类型读者 (学生和教师) 提供借阅服务。不同读者类型有不同的借阅数量上限和借阅期限, 不同馆藏类型的借阅期限和逾期罚款标准也不同。系统需在统一管理的同时正确处理这些差异化规则。

#### 2.1.2 功能需求

1. 支持至少两种读者类型 (学生、教师), 权限不同
2. 支持至少两种馆藏类型 (图书、杂志), 借阅规则不同
3. 读者可借阅馆藏物品, 系统自动记录借阅信息

4. 归还时自动判断是否逾期，根据物品类型计算罚款
5. 支持对读者和馆藏的增删查改
6. 具有明确的退出条件（用户选择退出菜单）
7. 使用面向对象设计，通过继承和多态实现差异化行为

### 2.1.3 用例分析

**用例名称：**学生借阅图书

**参与者：**学生（读者）

**前置条件：**(1) 学生已在系统中注册；(2) 目标图书在馆藏中存在且处于”在库”状态；(3) 学生当前借阅数量未达到上限（5 本）。

**主事件流：**1. 管理员选择”借阅管理 → 借书”；2. 输入学生学号；3. 系统查找学生，确认身份为”学生”，借阅上限 5 本，借期 30 天；4. 系统检查当前借阅数量，未超限则继续；5. 输入目标图书 ISBN；6. 系统查找图书，确认状态为”在库”；7. 系统将图书状态设为”已借出”，生成借阅记录（包含借出日期和应还日期 = 借出日期 + 30 天）；8. 系统显示借阅成功信息及应还日期。

**后置条件：**图书状态变为”已借出”，系统中新增一条借阅记录。

## 2.2 总体设计

### 2.2.1 类继承结构图

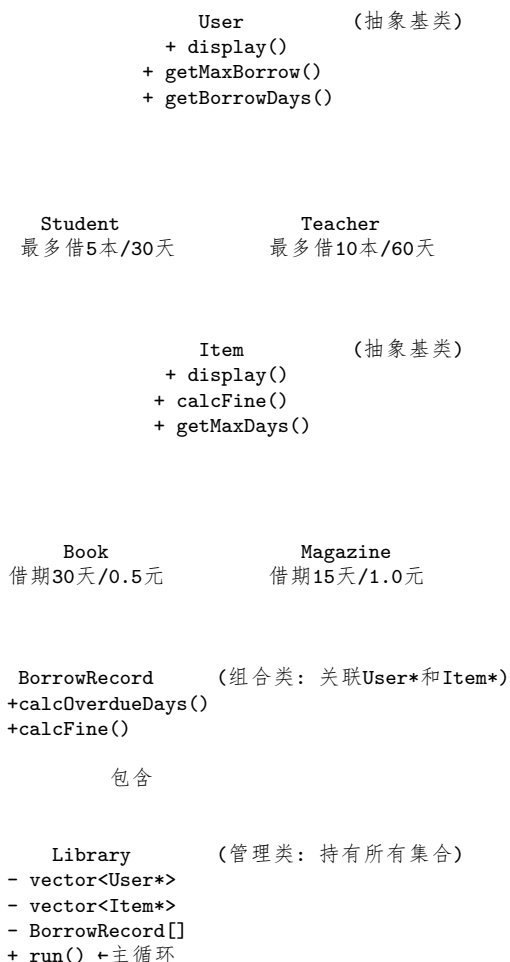


图 3: 类继承结构图：表示继承（空心三角箭头指向基类），- 表示组合关系

图中展示了全部 8 个类的名称和关系：(1) 继承关系：User → Student/Teacher, Item → Book/-Magazine；(2) 组合关系：Library 持有 User\*、Item\* 集合及 BorrowRecord 集合，BorrowRecord 关联 User\* 和 Item\*。

### 2.2.2 类职责说明

表 1: 类职责说明

| 类名           | 类型   | 核心职责                                                         |
|--------------|------|--------------------------------------------------------------|
| User         | 抽象基类 | 定义读者公共接口: display()、getMaxBorrow()、getBorrowDays()、getRole() |
| Student      | 派生类  | 学生借阅规则 (5 本上限、30 天借期), 存储年级和专业                               |
| Teacher      | 派生类  | 教师借阅规则 (10 本上限、60 天借期), 存储职称和院系                              |
| Item         | 抽象基类 | 定义馆藏公共接口: display()、getMaxDays()、calcFine()、getType()        |
| Book         | 派生类  | 图书借阅规则 (30 天上限、0.5 元/天罚款), 存储出版社和页数                          |
| Magazine     | 派生类  | 杂志借阅规则 (15 天上限、1.0 元/天罚款), 存储期号和月份                           |
| BorrowRecord | 组合类  | 记录单次借阅信息, 关联借阅者和物品, 计算逾期和罚款                                  |
| Library      | 管理类  | 持有所有数据集合, 提供全部业务操作, 含主菜单循环                                   |

### 2.2.3 继承策略分析

本项目采用双继承树结构, 分别对“读者”和“馆藏”两个业务维度建立基类-派生类体系。

**选择此结构的原因:** (1) 业务差异化需求驱动——不同类型读者和物品有各自不同的业务规则 (借阅数量、期限、罚款费率), 虚函数让每个派生类“自带”规则, 外部通过基类指针调用即可。例如 `u->getMaxBorrow()` 对 `Student` 返回 5, 对 `Teacher` 返回 10。(2) 符合开闭原则——新增读者类型 (如“访客”) 或馆藏类型只需加派生类重写虚函数, `Library` 代码零修改。(3) 代码复用——基类封装公共属性和行为, 避免重复定义。

**访问控制与虚函数设计:** 成员变量 (`name`、`id`、`isbn` 等) 设为 `protected`, 派生类可直接访问 (如在 `display()` 中直接使用 `name`), 同时阻止外部随意访问。`display()`、`getMaxBorrow()`、`calcFine()` 等设计为纯虚函数, 原因: (1) 基类无法确定具体规则, 行为必须由派生类填充; (2) 纯虚函数强制派生类实现, 避免遗漏; (3) `Library` 通过 `vector<User*>` 和 `vector<Item*>` 统一管理, 多态自动分发到正确实现。

## 2.3 详细设计

### 2.3.1 关键类的接口设计

选取 `User` (基类)、`Student` (派生类)、`Item` (基类) 三个核心类:

Listing 1: User 抽象基类 (User.h)

```

1 class User {
2 protected:
3     std::string name, id, phone;
4 public:
5     User(const std::string& n, const std::string& i,
6         const std::string& p);
7     virtual ~User() {}
8     virtual void display() const = 0;           // 纯虚
9     virtual int getMaxBorrow() const = 0;       // 纯虚
10    virtual int getBorrowDays() const = 0;      // 纯虚
11    virtual std::string getRole() const = 0;     // 纯虚
12    std::string getId() const { return id; }
13    std::string getName() const { return name; }
14 };

```

Listing 2: Student 派生类 (Student.h)

```

1 class Student : public User {

```

```

2 private:
3     std::string grade, major;
4 public:
5     Student(const std::string& n, const std::string& i,
6             const std::string& p, const std::string& g,
7             const std::string& m);
8     void display() const override;
9     int getMaxBorrow() const override;    // 返回 5
10    int getBorrowDays() const override;    // 返回 30
11    std::string getRole() const override;  // 返回"学生"
12 };

```

Listing 3: Item 抽象基类 (Item.h)

```

1 class Item {
2 protected:
3     std::string title, isbn, author;
4     int year;
5     bool available;
6 public:
7     Item(const std::string& t, const std::string& i,
8          const std::string& a, int y);
9     virtual ~Item() {}
10    virtual void display() const = 0;
11    virtual std::string getType() const = 0;
12    virtual int getMaxDays() const = 0;
13    virtual double calcFine(int overdueDays) const = 0;
14    bool isAvailable() const { return available; }
15    void setAvailable(bool a) { available = a; }
16 };

```

### 2.3.2 虚函数与多态的体现

以借书操作 borrowItem() 中的真实代码说明多态如何工作：

Listing 4: Library.cpp borrowItem()——多态调用

```

1 void Library::borrowItem() {
2     User* u = findUser(userId);    // User*, 可能是 Student 或 Teacher
3     Item* it = findItem(isbn);    // Item*, 可能是 Book 或 Magazine
4
5     // 多态: Student 返回 5, Teacher 返回 10
6     if (countBorrowedBy(userId) >= u->getMaxBorrow()) {
7         cout << u->getRole() << "已达借阅上限!" << endl;
8         return;
9     }
10    // 多态: Student 返回 30, Teacher 返回 60
11    int days = u->getBorrowDays();
12    records.push_back(BorrowRecord(u, it, today, days));
13 }

```

**关键点：**findUser() 返回 User\* 类型，调用者不知道具体是 Student 还是 Teacher。通过虚函数机制，u->getMaxBorrow() 在运行时自动根据对象真实类型调用正确函数。这体现了上层逻辑依赖抽象接口，具体行为由下层实现决定的核心原则。

显示所有读者时同样体现多态：

```

1 for (auto u : users) {
2     u->display();    // Student 和 Teacher 各有不同的显示格式
3 }

```

users 是 vector<User\*>，循环只需调用 display() 虚函数，多态自动选择正确版本。新增读者类型时，此循环无需任何修改。

### 2.3.3 核心业务逻辑：借阅-归还流程

系统最复杂的业务流程是“借书 → 还书 → 罚款计算”，涉及多个类的协作：



**借书阶段：**Library 收到读者编号和 ISBN → findUser() 获取 User\* → 调用 getMaxBorrow()（多态）检查上限 → findItem() 获取 Item\* → 检查 available 状态 → 调用 getBorrowDays()（多态）确定借期 → 创建 BorrowRecord，计算应还日期 → 将 available 设为 false。

**还书阶段：**Library 收到 ISBN → 在 records 中查找该物品的未归还记录 → markReturned() → 将 Item 的 available 设回 true → 调用 calcFine()，内部通过 Item\* 多态调用 calcFine()——Book 返回逾期天数 × 0.5 元，Magazine 返回逾期天数 × 1.0 元。

整个流程中 Library 只与基类指针交互，完全依赖虚函数多态实现差异化行为。

### 3 核心难点与解决方案

#### 3.1 问题描述

开发中最棘手的问题是悬空指针导致程序崩溃。BorrowRecord 持有 User\* borrower 和 Item\* item 两个裸指针，指向的对象由 Library 统一管理。当用户删除读者或馆藏时，对应对象被 delete 释放，但 BorrowRecord 中的指针没有同步更新，变成悬空指针——后续访问即崩溃。更隐蔽的是：删除前只检查“未归还”记录并拒绝删除，但已归还的历史记录中的指针仍指向被删对象，稍后查看历史记录时就触发访问冲突。

#### 3.2 定位过程

(1) 析构函数中加日志打印每个 delete 的地址；(2) 分场景测试：不删数据/正常删/删后操作，发现崩溃只出现在涉及删除的场景；(3) 崩溃地址与已 delete 地址匹配——确认二次释放或悬空指针；(4) 追踪 BorrowRecord 发现其指针未被纳入删除检查——锁定问题。

#### 3.3 解决方案

核心思路：删除前检查所有关联记录（含已归还），有关联则拒绝删除。

Listing 5: deleteUser()——检查所有关联记录

```

1 void Library::deleteUser() {
2     for (auto it = users.begin(); it != users.end(); ++it) {
3         if ((*it)->getId() == id) {
4             bool hasRecords = false;
5             for (auto& r : records) {
6                 if (r.getBorrower()->getId() == id) {
7                     hasRecords = true; break;
8                 }
9             }
10            if (hasRecords) {
11                cout << "该读者存在借阅记录，无法删除！" << endl;
12                return; // 拒绝删除，而非崩溃
13            }
14            delete *it; users.erase(it);
15        }
16    }
17 }

```

选择“拒绝删除”而非“级联删除”的原因：(1) 历史记录有保留价值；(2) 有未归还物品时删除读者缺乏合理业务语义。对于课程设计规模此方案合适；更大系统可考虑 std::shared\_ptr。

### 4 编译与运行说明

#### 4.1 开发环境与编译

- 操作系统：Windows 11 Pro 64-bit
- 编译器：g++ (MinGW-w64) 13.2.0 或 Visual Studio 2022 (MSVC v143)
- C++ 标准：C++11 (-std=c++11)
- 第三方依赖：无，仅使用 C++ 标准库

g++ 编译命令：

```

1 g++ -std=c++11 -Wall -o library_system.exe \
2     main.cpp User.cpp Student.cpp Teacher.cpp \
3     Item.cpp Book.cpp Magazine.cpp \
4     BorrowRecord.cpp Library.cpp

```

**Visual Studio 2022:** 文件 → 打开 → CMake → 选择 CMakeLists.txt → Ctrl+F5。

## 4.2 运行示例

启动后显示主菜单，输入数字选择操作，输入 0 退出系统：

图 书 借 阅 管 理 系 统

1. 图书管理    2. 读者管理
3. 借阅管理    0. 退出系统

## 5 总结与心得

### 5.1 项目完成情况

**已完成：**图书和杂志的增删查改、学生和教师的增删查改、借书操作（检查上限和库存）、还书操作（自动计算逾期和罚款）、逾期记录查询、系统退出时释放所有动态内存、双继承树设计（4 个派生类全部重写基类虚函数）、Library 通过基类指针统一管理所有对象体现多态。

**未完成/可改进：**数据持久化（关闭程序数据丢失）、更精确的日期计算（目前近似每月 30 天）、未增加管理员角色区分权限。

### 5.2 对面向对象设计的理解深化

完成本项目最大的收获是对“**依赖抽象而非具体**”这一原则有了切身体会。最初写 borrowItem() 时曾试图用 if-else 判断读者类型——“如果是学生限制 5 本，是教师限制 10 本”。但这样每加一种读者类型就要改多处代码。改为虚函数后 Library 只需调用 u->getMaxBorrow()，具体返回值由对象实际类型决定——**变化的部分被封装在派生类内部，上层代码保持稳定**。

另外体会到继承不是越多越好。曾考虑创建“教职工”中间层作为 Teacher 和可能的管理员的基类，但当前需求下属于过度设计。继承深度的控制也是 OOP 的学问。

### 5.3 收获与改进方向

**收获：**实践了从需求分析到类图设计到编码实现的完整流程；理解了虚函数和多态机制；学会了指针环境下的对象生命周期管理；体会到前期设计对后期编码效率的影响。

**改进方向：**增加文件读写实现数据持久化；增加管理员角色；使用 Qt 添加图形界面；增加图书预约功能。

## A 附录：完整源代码

本项目全部源代码（17 个文件）托管在 GitHub：

<https://github.com/hhh6996/cpp-library-management-system>

### A.1 文件清单

|                                   |                   |
|-----------------------------------|-------------------|
| User.h / User.cpp                 | 读者抽象基类            |
| Student.h / Student.cpp           | 学生派生类（5 本/30 天）   |
| Teacher.h / Teacher.cpp           | 教师派生类（10 本/60 天）  |
| Item.h / Item.cpp                 | 馆藏抽象基类            |
| Book.h / Book.cpp                 | 图书派生类（30 天/0.5 元） |
| Magazine.h / Magazine.cpp         | 杂志派生类（15 天/1.0 元） |
| BorrowRecord.h / BorrowRecord.cpp | 借阅记录 + 日期工具       |
| Library.h / Library.cpp           | 系统管理核心            |
| main.cpp                          | 程序入口              |

### A.2 头文件

Listing 6: User.h

```
1 #pragma once
2 #include <string>
```

```

3 #include <iostream>
4
5 // 用户基类（抽象类）
6 class User {
7 protected:
8     std::string name;    // 姓名
9     std::string id;      // 编号（学号/工号）
10    std::string phone;   // 电话
11
12 public:
13     User(const std::string& name, const std::string& id, const std::string& phone);
14     virtual ~User() {}
15
16     // 纯虚函数 —— 派生类必须实现
17     virtual void display() const = 0;
18     virtual int getMaxBorrow() const = 0;    // 最多可借几本
19     virtual int getBorrowDays() const = 0;   // 每本可借多少天
20     virtual std::string getRole() const = 0; // 角色名称：学生/教师
21
22     // 普通成员函数
23     std::string getId() const { return id; }
24     std::string getName() const { return name; }
25 };

```

Listing 7: Student.h

```

1 #pragma once
2 #include "User.h"
3
4 // 学生类 —— 继承自 User
5 class Student : public User {
6 private:
7     std::string grade;    // 年级，如 "2024级"
8     std::string major;    // 专业，如 "计算机科学与技术"
9
10 public:
11     Student(const std::string& name, const std::string& id,
12             const std::string& phone, const std::string& grade,
13             const std::string& major);
14
15     // 重写基类虚函数
16     void display() const override;
17     int getMaxBorrow() const override;    // 学生最多借 5 本
18     int getBorrowDays() const override;   // 学生可借 30 天
19     std::string getRole() const override; // 返回 "学生"
20 };

```

Listing 8: Teacher.h

```

1 #pragma once
2 #include "User.h"
3
4 // 教师类 —— 继承自 User
5 class Teacher : public User {
6 private:
7     std::string title;    // 职称，如 "教授"
8     std::string department; // 院系
9
10 public:
11     Teacher(const std::string& name, const std::string& id,
12             const std::string& phone, const std::string& title,
13             const std::string& department);
14
15     void display() const override;
16     int getMaxBorrow() const override;    // 教师最多借 10 本
17     int getBorrowDays() const override;   // 教师可借 60 天
18     std::string getRole() const override; // 返回 "教师"

```

19 };

Listing 9: Item.h

```

1 #pragma once
2 #include <string>
3 #include <iostream>
4
5 // 馆藏物品基类（抽象类）
6 class Item {
7 protected:
8     std::string title;    // 书名/刊名
9     std::string isbn;    // ISBN / 期刊号
10    std::string author;   // 作者/主编
11    int year;             // 出版年份
12    bool available;       // 是否在库（未被借出）
13
14 public:
15     Item(const std::string& title, const std::string& isbn,
16          const std::string& author, int year);
17     virtual ~Item() {}
18
19     // 纯虚函数
20     virtual void display() const = 0;           // 显示信息
21     virtual std::string getType() const = 0;    // 类型名称：图书/杂志
22     virtual int getMaxDays() const = 0;         // 最多可借多少天
23     virtual double calcFine(int overdueDays) const = 0; // 逾期罚款计算
24
25     // 普通函数
26     std::string getIsbn() const { return isbn; }
27     std::string getTitle() const { return title; }
28     std::string getAuthor() const { return author; }
29     bool isAvailable() const { return available; }
30     void setAvailable(bool a) { available = a; }
31 };

```

Listing 10: Book.h

```

1 #pragma once
2 #include "Item.h"
3
4 // 图书类 —— 继承自 Item
5 class Book : public Item {
6 private:
7     std::string publisher; // 出版社
8     int pages;             // 页数
9
10 public:
11     Book(const std::string& title, const std::string& isbn,
12          const std::string& author, int year,
13          const std::string& publisher, int pages);
14
15     void display() const override;
16     std::string getType() const override;
17     int getMaxDays() const override;           // 图书可借 30 天
18     double calcFine(int overdueDays) const override; // 逾期 0.5 元/天
19 };

```

Listing 11: Magazine.h

```

1 #pragma once
2 #include "Item.h"
3
4 // 杂志类 —— 继承自 Item
5 class Magazine : public Item {
6 private:
7     int issueNo; // 期号

```

```

8     int month;          // 发行月份
9
10 public:
11     Magazine(const std::string& title, const std::string& isbn,
12             const std::string& author, int year,
13             int issueNo, int month);
14
15     void display() const override;
16     std::string getType() const override;
17     int getMaxDays() const override;          // 杂志可借 15 天
18     double calcFine(int overdueDays) const override; // 逾期 1.0 元/天
19 };

```

Listing 12: BorrowRecord.h

```

1 #pragma once
2 #include <string>
3
4 // 简单的日期结构体
5 struct Date {
6     int year;
7     int month;
8     int day;
9 };
10
11 class User;
12 class Item;
13
14 // 借阅记录类
15 class BorrowRecord {
16 private:
17     User* borrower;      // 借阅者 (指针, 指向 User 派生类对象)
18     Item* item;          // 借阅物品 (指针, 指向 Item 派生类对象)
19     Date borrowDate;     // 借出日期
20     Date dueDate;        // 应还日期
21     bool returned;       // 是否已归还
22
23 public:
24     BorrowRecord(User* u, Item* it, const Date& bDate, int borrowDays);
25
26     void display() const;          // 显示借阅记录
27     int calcOverdueDays(const Date& today) const; // 计算逾期天数
28     double calcFine(const Date& today) const;    // 计算罚款金额
29     void markReturned() { returned = true; }
30
31     bool isReturned() const { return returned; }
32     User* getBorrower() const { return borrower; }
33     Item* getItem() const { return item; }
34     std::string getBorrowerName() const;
35     std::string getItemTitle() const;
36     std::string getDueDateStr() const; // 格式化应还日期字符串
37
38     // 日期工具函数
39     static Date getCurrentDate();          // 获取当前日期
40     static int daysBetween(const Date& d1, const Date& d2); // 两个日期相差天数
41 };

```

Listing 13: Library.h

```

1 #pragma once
2 #include <vector>
3 #include "User.h"
4 #include "Item.h"
5 #include "BorrowRecord.h"
6
7 // 图书馆管理类 —— 系统核心
8 class Library {

```

```

9 private:
10     std::vector<User*> users;           // 所有读者（多态指针）
11     std::vector<Item*> items;           // 所有馆藏（多态指针）
12     std::vector<BorrowRecord> records; // 所有借阅记录
13
14     // 内部查找辅助
15     User* findUser(const std::string& id);
16     Item* findItem(const std::string& isbn);
17     int countBorrowedBy(const std::string& userId); // 某读者当前借了几本
18
19     // 子菜单
20     void bookMenu(); // 图书管理
21     void userMenu(); // 读者管理
22     void borrowMenu(); // 借阅管理
23
24     // 具体操作函数
25     void addBook();
26     void addMagazine();
27     void deleteItem();
28     void searchItems();
29     void listAllItems();
30
31     void addStudent();
32     void addTeacher();
33     void deleteUser();
34     void listAllUsers();
35
36     void borrowItem();
37     void returnItem();
38     void listRecords();
39     void listOverdue();
40
41 public:
42     ~Library();
43     void run(); // 启动主循环
44 };

```

### A.3 实现文件

Listing 14: User.cpp

```

1 #include "User.h"
2
3 User::User(const std::string& n, const std::string& i, const std::string& p)
4     : name(n), id(i), phone(p) {}

```

Listing 15: Student.cpp

```

1 #include "Student.h"
2 #include <iomanip>
3
4 Student::Student(const std::string& n, const std::string& i,
5                 const std::string& p, const std::string& g,
6                 const std::string& m)
7     : User(n, i, p), grade(g), major(m) {}
8
9 void Student::display() const {
10     using namespace std;
11     cout << "      学生信息      " << endl;
12     cout << "  姓名: " << setw(20) << left << name << "  " << endl;
13     cout << "  学号: " << setw(20) << left << id << "  " << endl;
14     cout << "  电话: " << setw(20) << left << phone << "  " << endl;
15     cout << "  年级: " << setw(20) << left << grade << "  " << endl;
16     cout << "  专业: " << setw(20) << left << major << "  " << endl;
17     cout << "  身份: 学生      " << endl;
18     cout << "  最多可借: 5 本, 借期 30 天      " << endl;

```

```

19     cout << "                                " << endl;
20 }
21
22 int Student::getMaxBorrow() const { return 5; }
23
24 int Student::getBorrowDays() const { return 30; }
25
26 std::string Student::getRole() const { return "学生"; }

```

Listing 16: Teacher.cpp

```

1 #include "Teacher.h"
2 #include <iomanip>
3
4 Teacher::Teacher(const std::string& n, const std::string& i,
5                 const std::string& p, const std::string& t,
6                 const std::string& d)
7     : User(n, i, p), title(t), department(d) {}
8
9 void Teacher::display() const {
10     using namespace std;
11     cout << "          教师信息          " << endl;
12     cout << " 姓名: " << setw(20) << left << name << " " << endl;
13     cout << " 工号: " << setw(20) << left << id << " " << endl;
14     cout << " 电话: " << setw(20) << left << phone << " " << endl;
15     cout << " 职称: " << setw(20) << left << title << " " << endl;
16     cout << " 院系: " << setw(20) << left << department << " " << endl;
17     cout << " 身份: 教师          " << endl;
18     cout << " 最多可借: 10 本, 借期 60 天 " << endl;
19     cout << "                                " << endl;
20 }
21
22 int Teacher::getMaxBorrow() const { return 10; }
23
24 int Teacher::getBorrowDays() const { return 60; }
25
26 std::string Teacher::getRole() const { return "教师"; }

```

Listing 17: Item.cpp

```

1 #include "Item.h"
2
3 Item::Item(const std::string& t, const std::string& i,
4           const std::string& a, int y)
5     : title(t), isbn(i), author(a), year(y), available(true) {}

```

Listing 18: Book.cpp

```

1 #include "Book.h"
2 #include <iomanip>
3
4 Book::Book(const std::string& t, const std::string& i,
5           const std::string& a, int y,
6           const std::string& pub, int pg)
7     : Item(t, i, a, y), publisher(pub), pages(pg) {}
8
9 void Book::display() const {
10     using namespace std;
11     cout << "          图书信息          " << endl;
12     cout << " 书名: " << setw(20) << left << title << " " << endl;
13     cout << " ISBN: " << setw(20) << left << isbn << " " << endl;
14     cout << " 作者: " << setw(20) << left << author << " " << endl;
15     cout << " 出版社: " << setw(18) << left << publisher << " " << endl;
16     cout << " 年份: " << setw(20) << left << year << " " << endl;
17     cout << " 页数: " << setw(20) << left << pages << " " << endl;
18     cout << " 状态: " << (available ? "在库" : "已借出") << " " << endl;

```



```

19     cout << "    类型: 图书, 借期 30 天" << endl;
20     cout << "                                " << endl;
21 }
22
23 std::string Book::getType() const { return "图书"; }
24
25 int Book::getMaxDays() const { return 30; }
26
27 double Book::calcFine(int overdueDays) const {
28     if (overdueDays <= 0) return 0.0;
29     return overdueDays * 0.5; // 图书逾期每天 0.5 元
30 }

```

Listing 19: Magazine.cpp

```

1  #include "Magazine.h"
2  #include <iomanip>
3
4  Magazine::Magazine(const std::string& t, const std::string& i,
5                    const std::string& a, int y,
6                    int issue, int m)
7      : Item(t, i, a, y), issueNo(issue), month(m) {}
8
9  void Magazine::display() const {
10     using namespace std;
11     cout << "        杂志信息" << endl;
12     cout << "    刊名: " << setw(20) << left << title << " " << endl;
13     cout << "    刊号: " << setw(20) << left << isbn << " " << endl;
14     cout << "    主编: " << setw(20) << left << author << " " << endl;
15     cout << "    年份: " << setw(20) << left << year << " " << endl;
16     cout << "    期号: 第" << setw(18) << left << issueNo << " " << endl;
17     cout << "    月份: " << setw(20) << left << month << " " << endl;
18     cout << "    状态: " << (available ? "在库" : "已借出") << " " <<
19         endl;
20     cout << "    类型: 杂志, 借期 15 天" << endl;
21     cout << "                                " << endl;
22 }
23
24 std::string Magazine::getType() const { return "杂志"; }
25
26 int Magazine::getMaxDays() const { return 15; }
27
28 double Magazine::calcFine(int overdueDays) const {
29     if (overdueDays <= 0) return 0.0;
30     return overdueDays * 1.0; // 杂志逾期每天 1.0 元
31 }

```

Listing 20: BorrowRecord.cpp

```

1  #include "BorrowRecord.h"
2  #include "User.h"
3  #include "Item.h"
4  #include <iostream>
5  #include <iomanip>
6  #include <ctime>
7
8  BorrowRecord::BorrowRecord(User* u, Item* it, const Date& bDate, int borrowDays)
9      : borrower(u), item(it), borrowDate(bDate), returned(false)
10 {
11     // 根据借出日期和可借天数计算应还日期
12     dueDate = borrowDate;
13     dueDate.day += borrowDays;
14
15     // 简化月份进位处理
16     int daysInMonth[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
17     while (dueDate.day > daysInMonth[dueDate.month]) {
18         dueDate.day -= daysInMonth[dueDate.month];

```



```

19     dueDate.month++;
20     if (dueDate.month > 12) {
21         dueDate.month = 1;
22         dueDate.year++;
23     }
24 }
25 }
26
27 void BorrowRecord::display() const {
28     using namespace std;
29     cout << "借 阅 人: " << getBorrowerName()
30         << " | 物 品: " << getItemTitle()
31         << " | 借 出: " << borrowDate.year << "-" << borrowDate.month << "-" <<
            borrowDate.day
32         << " | 应 还: " << dueDate.year << "-" << dueDate.month << "-" << dueDate.day
33         << " | 状 态: " << (returned ? "已还" : "未还") << endl;
34 }
35
36 int BorrowRecord::calcOverdueDays(const Date& today) const {
37     if (returned) return 0;
38     int diff = daysBetween(dueDate, today);
39     return diff > 0 ? diff : 0; // 未逾期返回 0
40 }
41
42 double BorrowRecord::calcFine(const Date& today) const {
43     int overdue = calcOverdueDays(today);
44     if (overdue <= 0) return 0.0;
45     return item->calcFine(overdue); // 多态调用: 不同物品罚款不同
46 }
47
48 std::string BorrowRecord::getBorrowerName() const {
49     return borrower->getName();
50 }
51
52 std::string BorrowRecord::getItemTitle() const {
53     return item->getTitle();
54 }
55
56 std::string BorrowRecord::getDueDateStr() const {
57     return std::to_string(dueDate.year) + "-" +
58         std::to_string(dueDate.month) + "-" +
59         std::to_string(dueDate.day);
60 }
61
62 // ----- 日期工具函数 -----
63
64 Date BorrowRecord::getCurrentDate() {
65     time_t now = time(0);
66     tm* ltm = localtime(&now);
67     Date d;
68     d.year = 1900 + ltm->tm_year;
69     d.month = 1 + ltm->tm_mon;
70     d.day = ltm->tm_mday;
71     return d;
72 }
73
74 int BorrowRecord::daysBetween(const Date& d1, const Date& d2) {
75     // 简易天数计算: 近似每月 30 天
76     // 用于演示目的, 不考虑精确闰年
77     int total1 = d1.year * 365 + d1.month * 30 + d1.day;
78     int total2 = d2.year * 365 + d2.month * 30 + d2.day;
79     return total2 - total1;
80 }

```

Listing 21: Library.cpp

```

1 #include "Library.h"
2 #include "Student.h"
3 #include "Teacher.h"
4 #include "Book.h"
5 #include "Magazine.h"
6 #include <iostream>
7 #include <iomanip>
8 #include <algorithm>
9
10 using namespace std;
11
12 // ===== 辅助查找函数 =====
13
14 User* Library::findUser(const string& id) {
15     for (auto u : users) {
16         if (u->getId() == id) return u;
17     }
18     return nullptr;
19 }
20
21 Item* Library::findItem(const string& isbn) {
22     for (auto it : items) {
23         if (it->getIsbn() == isbn) return it;
24     }
25     return nullptr;
26 }
27
28 int Library::countBorrowedBy(const string& userId) {
29     int cnt = 0;
30     for (auto& r : records) {
31         if (!r.isReturned() && r.getBorrower()->getId() == userId) {
32             cnt++;
33         }
34     }
35     return cnt;
36 }
37
38 // ===== 析构函数 =====
39
40 Library::~Library() {
41     // 释放所有动态分配的 User 和 Item 对象
42     for (auto u : users) delete u;
43     for (auto it : items) delete it;
44 }
45
46 // ===== 主菜单循环 =====
47
48 void Library::run() {
49     int choice;
50     while (true) {
51         cout << "\n";
52         cout << "                                " << endl;
53         cout << "                图 书 借 阅 管 理 系 统                " << endl;
54         cout << "                                " << endl;
55         cout << "    1. 图书管理                                " << endl;
56         cout << "    2. 读者管理                                " << endl;
57         cout << "    3. 借阅管理                                " << endl;
58         cout << "    0. 退出系统                                " << endl;
59         cout << "                                " << endl;
60         cout << "请选择操作: ";
61         cin >> choice;
62
63         if (cin.fail()) {
64             cin.clear();
65             cin.ignore(10000, '\n');
66             cout << "输入无效, 请重新输入!" << endl;

```

```

67         continue;
68     }
69
70     switch (choice) {
71     case 1: bookMenu(); break;
72     case 2: userMenu(); break;
73     case 3: borrowMenu(); break;
74     case 0:
75         cout << "感谢使用，再见！" << endl;
76         return;
77     default:
78         cout << "无效选项，请重试。" << endl;
79     }
80 }
81 }
82
83 // ===== 图书管理子菜单 =====
84
85 void Library::bookMenu() {
86     int ch;
87     while (true) {
88         cout << "\n--- 图书管理 ---" << endl;
89         cout << "1. 添加图书" << endl;
90         cout << "2. 添加杂志" << endl;
91         cout << "3. 删除馆藏" << endl;
92         cout << "4. 查询馆藏" << endl;
93         cout << "5. 显示全部馆藏" << endl;
94         cout << "0. 返回主菜单" << endl;
95         cout << "请选择: ";
96         cin >> ch;
97
98         switch (ch) {
99         case 1: addBook(); break;
100        case 2: addMagazine(); break;
101        case 3: deleteItem(); break;
102        case 4: searchItems(); break;
103        case 5: listAllItems(); break;
104        case 0: return;
105        default: cout << "无效选项。" << endl;
106        }
107     }
108 }
109
110 void Library::addBook() {
111     string title, isbn, author, publisher;
112     int year, pages;
113
114     cin.ignore();
115     cout << "书名: "; getline(cin, title);
116     cout << "ISBN: "; getline(cin, isbn);
117
118     // 检查 ISBN 是否已存在
119     if (findItem(isbn)) {
120         cout << "该 ISBN 已存在，添加失败！" << endl;
121         return;
122     }
123
124     cout << "作者: "; getline(cin, author);
125     cout << "出版社: "; getline(cin, publisher);
126     cout << "出版年份: "; cin >> year;
127     cout << "页数: "; cin >> pages;
128
129     Book* b = new Book(title, isbn, author, year, publisher, pages);
130     items.push_back(b);
131     cout << "图书添加成功！" << endl;
132 }

```

```

133
134 void Library::addMagazine() {
135     string title, issn, editor;
136     int year, issueNo, month;
137
138     cin.ignore();
139     cout << "刊名: ";      getline(cin, title);
140     cout << "刊号: ";      getline(cin, issn);
141
142     if (findItem(issn)) {
143         cout << "该刊号已存在, 添加失败!" << endl;
144         return;
145     }
146
147     cout << "主编: ";      getline(cin, editor);
148     cout << "出版年份: ";  cin >> year;
149     cout << "期号: ";      cin >> issueNo;
150     cout << "月份: ";      cin >> month;
151
152     Magazine* m = new Magazine(title, issn, editor, year, issueNo, month);
153     items.push_back(m);
154     cout << "杂志添加成功!" << endl;
155 }
156
157 void Library::deleteItem() {
158     string keyword;
159     cin.ignore();
160     cout << "输入要删除的 ISBN/刊号: ";
161     getline(cin, keyword);
162
163     for (auto it = items.begin(); it != items.end(); ++it) {
164         if ((*it)->getIsbn() == keyword) {
165             // 检查是否有任何关联的借阅记录 (含已归还)
166             // 否则删除后 BorrowRecord 中的 Item* 会变成悬空指针
167             for (auto& r : records) {
168                 if (r.getItem() == *it) {
169                     cout << "该物品存在借阅记录, 无法删除!" << endl;
170                     return;
171                 }
172             }
173             delete *it;
174             items.erase(it);
175             cout << "删除成功!" << endl;
176             return;
177         }
178     }
179     cout << "未找到该物品。" << endl;
180 }
181
182 void Library::searchItems() {
183     string keyword;
184     cin.ignore();
185     cout << "输入搜索关键词 (书名/ISBN/作者): ";
186     getline(cin, keyword);
187
188     bool found = false;
189     for (auto it : items) {
190         if (it->getTitle().find(keyword) != string::npos ||
191             it->getIsbn().find(keyword) != string::npos ||
192             it->getAuthor().find(keyword) != string::npos) {
193             it->display();
194             found = true;
195         }
196     }
197     if (!found) cout << "未找到匹配的馆藏。" << endl;
198 }

```

```

199
200 void Library::listAllItems() {
201     if (items.empty()) {
202         cout << "馆藏为空。" << endl;
203         return;
204     }
205     cout << "\n==== 全部馆藏 (共 " << items.size() << " 件) =====> << endl;
206     for (auto it : items) {
207         it->display(); // 多态调用
208     }
209 }
210
211 // ===== 读者管理子菜单 =====
212
213 void Library::userMenu() {
214     int ch;
215     while (true) {
216         cout << "\n--- 读者管理 ---" << endl;
217         cout << "1. 添加学生" << endl;
218         cout << "2. 添加教师" << endl;
219         cout << "3. 删除读者" << endl;
220         cout << "4. 显示全部读者" << endl;
221         cout << "0. 返回主菜单" << endl;
222         cout << "请选择: ";
223         cin >> ch;
224
225         switch (ch) {
226             case 1: addStudent(); break;
227             case 2: addTeacher(); break;
228             case 3: deleteUser(); break;
229             case 4: listAllUsers(); break;
230             case 0: return;
231             default: cout << "无效选项。" << endl;
232         }
233     }
234 }
235
236 void Library::addStudent() {
237     string name, id, phone, grade, major;
238     cin.ignore();
239     cout << "姓名: "; getline(cin, name);
240     cout << "学号: "; getline(cin, id);
241
242     if (findUser(id)) {
243         cout << "该学号已存在!" << endl;
244         return;
245     }
246
247     cout << "电话: "; getline(cin, phone);
248     cout << "年级: "; getline(cin, grade);
249     cout << "专业: "; getline(cin, major);
250
251     users.push_back(new Student(name, id, phone, grade, major));
252     cout << "学生添加成功!" << endl;
253 }
254
255 void Library::addTeacher() {
256     string name, id, phone, title, dept;
257     cin.ignore();
258     cout << "姓名: "; getline(cin, name);
259     cout << "工号: "; getline(cin, id);
260
261     if (findUser(id)) {
262         cout << "该工号已存在!" << endl;
263         return;
264     }

```

```

265     cout << "电话: "; getline(cin, phone);
266     cout << "职称: "; getline(cin, title);
267     cout << "院系: "; getline(cin, dept);
268
269
270     users.push_back(new Teacher(name, id, phone, title, dept));
271     cout << "教师添加成功!" << endl;
272 }
273
274 void Library::deleteUser() {
275     string id;
276     cin.ignore();
277     cout << "输入要删除的读者编号: ";
278     getline(cin, id);
279
280     for (auto it = users.begin(); it != users.end(); ++it) {
281         if ((*it)->getId() == id) {
282             // 检查是否有任何关联的借阅记录 (含已归还)
283             bool hasRecords = false;
284             for (auto& r : records) {
285                 if (r.getBorrower()->getId() == id) {
286                     hasRecords = true;
287                     break;
288                 }
289             }
290             if (hasRecords) {
291                 cout << "该读者存在借阅记录, 无法删除!" << endl;
292                 return;
293             }
294             delete *it;
295             users.erase(it);
296             cout << "删除成功!" << endl;
297             return;
298         }
299     }
300     cout << "未找到该读者。" << endl;
301 }
302
303 void Library::listAllUsers() {
304     if (users.empty()) {
305         cout << "暂无读者。" << endl;
306         return;
307     }
308     cout << "\n==== 全部读者 (共 " << users.size() << " 人) =====> << endl;
309     for (auto u : users) {
310         u->display(); // 多态调用
311     }
312 }
313
314 // ===== 借阅管理子菜单 =====
315
316 void Library::borrowMenu() {
317     int ch;
318     while (true) {
319         cout << "\n--- 借阅管理 ---" << endl;
320         cout << "1. 借书" << endl;
321         cout << "2. 还书" << endl;
322         cout << "3. 查看借阅记录" << endl;
323         cout << "4. 逾期记录" << endl;
324         cout << "0. 返回主菜单" << endl;
325         cout << "请选择: ";
326         cin >> ch;
327
328         switch (ch) {
329             case 1: borrowItem(); break;
330             case 2: returnItem(); break;

```

```

331     case 3: listRecords(); break;
332     case 4: listOverdue(); break;
333     case 0: return;
334     default: cout << "无效选项。" << endl;
335 }
336 }
337 }
338
339 void Library::borrowItem() {
340     string userId, isbn;
341
342     if (users.empty()) {
343         cout << "系统中没有读者，请先添加读者。" << endl;
344         return;
345     }
346     if (items.empty()) {
347         cout << "馆藏为空，请先添加图书或杂志。" << endl;
348         return;
349     }
350
351     cin.ignore();
352     cout << "读者编号: "; getline(cin, userId);
353     User* u = findUser(userId);
354     if (!u) {
355         cout << "未找到该读者！" << endl;
356         return;
357     }
358
359     // 检查借阅数量上限
360     int current = countBorrowedBy(userId);
361     if (current >= u->getMaxBorrow()) { // 多态调用
362         cout << "借阅已达上限 (" << u->getRole() << "最多可借 "
363             << u->getMaxBorrow() << " 本)！" << endl;
364         return;
365     }
366
367     cout << "物品 ISBN/刊号: "; getline(cin, isbn);
368     Item* it = findItem(isbn);
369     if (!it) {
370         cout << "未找到该物品！" << endl;
371         return;
372     }
373     if (!it->isAvailable()) {
374         cout << "该物品已被借出！" << endl;
375         return;
376     }
377
378     // 执行借阅
379     it->setAvailable(false);
380     Date today = BorrowRecord::getCurrentDate();
381     int days = u->getBorrowDays(); // 多态调用
382     records.push_back(BorrowRecord(u, it, today, days));
383
384     cout << "借阅成功！" << endl;
385     cout << "借阅人: " << u->getName() << " (" << u->getRole() << ")" << endl;
386     cout << "物品: " << it->getTitle() << " (" << it->getType() << ")" << endl;
387     cout << "应还日期: " << records.back().getDueDateStr() << endl;
388 }
389
390 void Library::returnItem() {
391     string isbn;
392     cin.ignore();
393     cout << "归还物品 ISBN/刊号: ";
394     getline(cin, isbn);
395
396     // 查找该物品的未归还记录

```

```

397     for (auto& r : records) {
398         if (!r.isReturned() && r.getItem()->getIsbn() == isbn) {
399             r.markReturned();
400             r.getItem()->setAvailable(true);
401
402             Date today = BorrowRecord::getCurrentDate();
403             double fine = r.calcFine(today);
404             int overdue = r.calcOverdueDays(today);
405
406             cout << "归还成功!" << endl;
407             if (overdue > 0) {
408                 cout << "逾期 " << overdue << " 天, 需缴纳罚款: "
409                     << fine << " 元" << endl;
410             } else {
411                 cout << "按时归还, 无罚款。" << endl;
412             }
413             return;
414         }
415     }
416     cout << "未找到该物品的借阅记录, 或已归还。" << endl;
417 }
418
419 void Library::listRecords() {
420     if (records.empty()) {
421         cout << "暂无借阅记录。" << endl;
422         return;
423     }
424     cout << "\n==== 全部借阅记录 (共 " << records.size() << " 条) =====> << endl;
425     for (auto& r : records) {
426         r.display();
427     }
428 }
429
430 void Library::listOverdue() {
431     Date today = BorrowRecord::getCurrentDate();
432     bool found = false;
433
434     cout << "\n==== 逾期未还记录 =====> << endl;
435     for (auto& r : records) {
436         if (!r.isReturned() && r.calcOverdueDays(today) > 0) {
437             r.display();
438             cout << "    → 逾期 " << r.calcOverdueDays(today)
439                 << " 天, 罚款 " << r.calcFine(today) << " 元" << endl;
440             found = true;
441         }
442     }
443     if (!found) cout << "没有逾期记录。" << endl;
444 }

```

Listing 22: main.cpp

```

1  #include "Library.h"
2  #include <iostream>
3  #include <cstdlib>
4
5  int main() {
6      // 设置控制台编码为 UTF-8 (Windows 下显示中文)
7      #ifdef _WIN32
8          system("chcp 65001 > nul");
9      #endif
10
11      std::cout << "===== " << std::endl;
12      std::cout << "          欢迎使用图书借阅管理系统 V1.0          " << std::endl;
13      std::cout << "===== " << std::endl;
14
15      Library lib;

```



```
16     lib.run();  
17  
18     return 0;  
19 }
```